

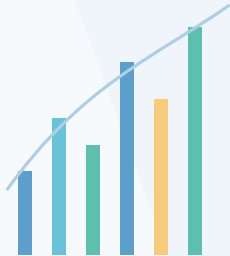


BIG DATA ANALYTICS IN FINANCE

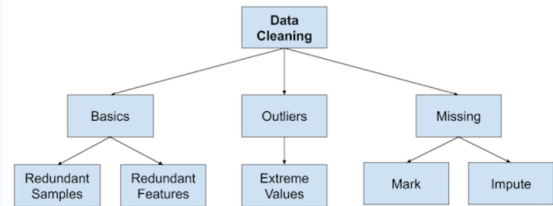
Lecture 2: Data Cleaning

Data integrity, outliers, missing values, and leakage-safe pipelines

Dr. Nguyen Tuan Long National Economics University | 2026



Overview: Data Cleaning



Four-part learning path

1. **Basic cleaning:** redundant samples and features.
2. **Outliers:** standard deviation, IQR, and Local Outlier Factor.
3. **Missing data:** removal and three imputation approaches.
4. **Automation:** leakage-safe Scikit-learn pipelines.

Teaching sequence used throughout the lecture

Concept → toy dataset → hand calculation → tool → implementation → full-data practice.

Contents

What We Will Cover Today

1. Why data cleaning is crucial
2. Basic data cleaning: first steps
3. Handling outliers
4. Handling missing data

From Cleaning to Production

5. Automating with Scikit-learn pipelines
6. Labs using full datasets
7. Summary and key takeaways

Why Is Data Cleaning Crucial?

Garbage In, Garbage Out

- ▶ Machine-learning models learn patterns from the data they receive.
- ▶ Flawed, noisy, or incorrect data leads to unreliable models.
- ▶ Cleaning creates a reliable foundation for model training.

Impact of clean data

- ▶ Improved performance and accuracy
- ▶ More robust, generalizable models
- ▶ Fewer errors during training
- ▶ More credible decisions

Goal of data cleaning

Create a high-quality dataset whose values, rows, and features correctly represent the problem.

Part 1 – Basic Data Cleaning

PART 1 Foundational Steps for Data Integrity

- ▶ **Remove zero-variance features:** policy fields with the same value for every application.
- ▶ **Remove duplicate rows:** repeated loan applications that can bias a credit-risk model.

```
import pandas as pd
import numpy as np
import io
```

Zero-Variance Features

Concept

A feature whose values are identical across all observations contains no information for separating or predicting observations.

Population variance

$$\text{Var}(X) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

If every $x_i = c$, then $\mu = c$ and every squared deviation is zero.

Decision rule

Remove a feature when its variance is zero, or below a small threshold chosen for the modeling problem.

Toy Dataset: SME Credit Portfolio

A five-loan portfolio illustrates constant features and duplicate applications.

```
csv_credit = '''loan_amount,interest_rate,debt_ratio,
                branch_code,default_flag
100,8.5,0.35,1,0
150,9.2,0.48,1,1
120,8.8,0.40,1,0
200,10.1,0.55,1,1
150,9.2,0.48,1,1''' # Duplicate application

df_credit = pd.read_csv(
    io.StringIO(csv_credit)
)
```

DataFrame preview

	Amt	Rate	Debt	Branch	Def.
0	100	8.5	.35	1	0
1	150	9.2	.48	1	1
2	120	8.8	.40	1	0
3	200	10.1	.55	1	1
4	150	9.2	.48	1	1

Dataset notes

Amount is in USD thousands. branch_code is constant, and row 4 duplicates row 1.

Hand Calculation: Variance in the Credit Data

Constant branch code

$$b = (1, 1, 1, 1, 1), \quad \bar{b} = 1$$

$$\text{Var}(b) = \frac{(1 - 1)^2 + \dots + (1 - 1)^2}{5} = 0$$

Therefore `branch_code` should be removed.

Comparison: loan amount

$$x = (100, 150, 120, 200, 150), \quad \bar{x} = 144$$

$$\sum_{i=1}^5 (x_i - 144)^2 = 5720$$

$$\text{Var}(x) = \frac{5720}{5} = 1144$$

Loan amount varies and is retained.

Tool and Implementation: VarianceThreshold

Tool

`sklearn.feature_selection.
VarianceThreshold`

Implementation steps

1. Separate predictors from the target.
2. Fit with `threshold=0`.
3. Recover retained feature names.

```
from sklearn.feature_selection import VarianceThreshold

X_credit = df_credit.drop(columns=['default_flag'])
selector = VarianceThreshold(threshold=0)
X_selected = selector.fit_transform(X_credit)

retained_cols = selector.get_feature_names_out(
    input_features=X_credit.columns
)
print(retained_cols)
```

Constant Categorical Features: Concept to Tool

Concept

A categorical feature with the same value for every firm or customer has no predictive value.

Toy hand check

If $\text{country} = (VN, VN, VN, VN, VN)$, then

$$\text{nunique}(\text{country}) = 1.$$

The feature cannot distinguish credit applicants.

Tool and implementation

`Series.nunique()` counts distinct categories.

```
country = pd.Series(['VN'] * 5)
print(country.nunique()) # 1

if country.nunique() == 1:
    print('Remove country')
```

Duplicate Credit Applications: Hand Check

Concept

Repeated applications can overweight one borrower profile and leak the same case across training and test sets.

Manual comparison

row 1 = (150, 9.2, 0.48, 1, 1)

row 4 = (150, 9.2, 0.48, 1, 1)

All values match, so retain the first application and remove the second.

Tool and Implementation: Duplicate Removal

Pandas tools

- ▶ `duplicated()` marks repeated applications.
- ▶ `drop_duplicates()` removes them.
- ▶ `keep='first'` is the default rule.

```
print(
    'Duplicate applications:',
    df_credit.duplicated().sum()
)

credit_no_dup = df_credit.drop_duplicates()
print(credit_no_dup)
```

Lab 1: Credit-Risk Data Integrity Audit

Tasks

1. Load the German Credit dataset from OpenML.
2. Identify constant numerical and categorical features.
3. Count and remove duplicate applications.
4. Report the portfolio shape before and after cleaning.

```
from sklearn.datasets import fetch_openml

credit_g = fetch_openml(
    name='credit-g', version=1,
    as_frame=True
)
df_lab1 = credit_g.frame
df_lab1.head()
```

Part 2 – Handling Outliers

PART 2 Identifying and Managing Anomalies

What are outliers?

Observations that are significantly different from other observations.

- ▶ Incorrect transaction or valuation records
- ▶ Data-entry errors in prices or borrower attributes
- ▶ Genuine market shocks or unusually large exposures

Potential impact

Outliers can skew statistical summaries and disproportionately influence model parameters.

Toy Dataset: Real-Estate Valuation Data

The dataset represents housing-market fundamentals and property values; extreme valuations may be outliers.

```
csv_housing = '''CRIM,ZN,INDUS,CHAS,NOX,RM,AGE,DIS,RAD,TAX,PTRATIO,B,LSTAT,MEDV
0.00632,18,2.31,0,0.538,6.575,65.2,4.09,1,296,15.3,396.9,4.98,24
0.02731,0,7.07,0,0.469,6.421,78.9,4.9671,2,242,17.8,396.9,9.14,21.6
0.02729,0,7.07,0,0.469,7.185,61.1,4.9671,2,242,17.8,392.83,4.03,34.7
0.03237,0,2.18,0,0.458,6.998,45.8,6.0622,3,222,18.7,394.63,2.94,33.4
0.06905,0,2.18,0,0.458,7.147,54.2,6.0622,3,222,18.7,396.9,5.33,36.2
0.9,80,20,0,0.6,12,90,2,5,666,20,350,30,500'''

df_housing = pd.read_csv(io.StringIO(csv_housing))
print("Initial housing data:")
df_housing
```

Injected anomaly

The final row contains extreme values, including RM=12 and MEDV=500.

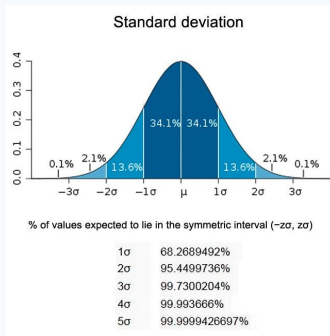
Outlier Detection: Standard Deviation Method

Concept

- ▶ Assumes a Gaussian or near-Gaussian distribution.
- ▶ Flags observations beyond a chosen number of standard deviations σ from mean μ .
- ▶ A common rule uses 3σ .

When to use

Use when the feature distribution is approximately normal.



Hand Calculation: Standard-Deviation Rule

Toy target values

$$MEDV = (24, 21.6, 34.7, 33.4, 36.2, 500)$$

Using the sample standard deviation:

$$\bar{x} = 108.32, \quad s = 191.98$$

Two-standard-deviation bounds

$$L = 108.32 - 2(191.98) = -275.64$$

$$U = 108.32 + 2(191.98) = 492.27$$

Since $500 > 492.27$, the final observation is flagged.

Caution

The outlier itself inflates both the mean and standard deviation.

Tool and Implementation: Standard Deviation

Pandas tools

- ▶ `Series.mean()`
- ▶ `Series.std()`
- ▶ Boolean masks for flagging and filtering

Toy-data setting

We use 2s so the injected anomaly is visible in the six-row sample.

```
data_col = df_housing['MEDV']
mean, std = data_col.mean(), data_col.std()
cut_off = std * 2
lower, upper = mean - cut_off, mean + cut_off

outliers = df_housing[
    (data_col < lower) | (data_col > upper)
]
print(f"Found {len(outliers)} outliers.")
print(outliers[['RM', 'MEDV']])

data_cleaned_outlier = df_housing[
    (data_col >= lower) & (data_col <= upper)
]
print(f"Original size: {len(df_housing)}")
print(f"Cleaned size: {len(data_cleaned_outlier)}")
```

Outlier Detection: Interquartile Range Method

Concept

A robust method that does not assume a particular distribution.

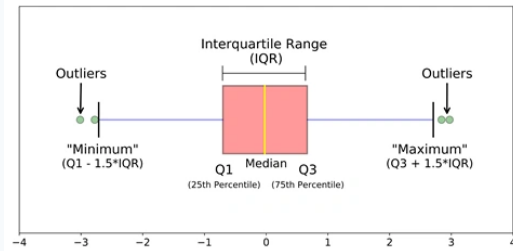
$$IQR = Q_3 - Q_1$$

$$Lower = Q_1 - 1.5IQR$$

$$Upper = Q_3 + 1.5IQR$$

When to use

Use for skewed data or when robustness to extreme values is important.



Hand Calculation: IQR Rule

Quartiles from the toy MEDV values

Sorted values:

21.6, 24, 33.4, 34.7, 36.2, 500

With Pandas' linear interpolation:

$$Q_1 = 26.35, \quad Q_3 = 35.825$$

$$IQR = 35.825 - 26.35 = 9.475$$

Outlier fences

$$L = 26.35 - 1.5(9.475) = 12.1375$$

$$U = 35.825 + 1.5(9.475) = 50.0375$$

Therefore $500 > 50.0375$ and is flagged as an outlier.

Interpretation

The IQR remains stable because it uses the middle 50% of the data.

Tool and Implementation: IQR

Pandas tools

- ▶ `Series.quantile()`
- ▶ Arithmetic for the two fences
- ▶ Boolean masks to locate outliers

```
data_col = df_housing['MEDV']

Q1 = data_col.quantile(0.25)
Q3 = data_col.quantile(0.75)
IQR = Q3 - Q1

lower_iqr = Q1 - 1.5 * IQR
upper_iqr = Q3 + 1.5 * IQR

outliers_iqr = df_housing[
    (data_col < lower_iqr) | (data_col > upper_iqr)
]
print(f"Found {len(outliers_iqr)} outliers using IQR.")
print(outliers_iqr[['RM', 'MEDV']])
```

Outlier Detection: Local Outlier Factor

Concept

- ▶ Unsupervised, model-based local-density method
- ▶ Useful for multivariate outliers
- ▶ Compares each point's local density with the densities of its neighbors
- ▶ A substantially larger LOF score indicates an isolated observation

Correct train/test usage

- ▶ Training data: use `fit_predict()` and remove detected outliers.
- ▶ Test data: use only `predict()`; do not fit again or remove test observations.

Leakage warning

Never fit LOF on the combined dataset before train/test splitting.

Hand Calculation: Local Outlier Factor

Consider one-dimensional points $A = 0.9$, $B = 1.0$, $C = 1.1$, and $D = 5.0$ with $k = 2$.

Local reachability densities

For D , the two nearest neighbors are C and B :

$$d(D, C) = 3.9, \quad d(D, B) = 4.0$$

Average reachability distance = $(3.9 + 4.0)/2 = 3.95$, so

$$LRD(D) = 1/3.95 = 0.253.$$

Compare with neighbor densities

Using the same definition:

$$LRD(C) = 6.667, \quad LRD(B) = 5.000$$

$$LOF(D) = \frac{6.667/0.253 + 5/0.253}{2} \approx 23.04$$

The score is far above 1, so D is locally isolated.

Meaning

LOF near 1 suggests similar density to neighboring observations.

Tool and Implementation: LocalOutlierFactor

Tool

`sklearn.neighbors.
LocalOutlierFactor`

Important parameters

- ▶ `n_neighbors=2`: used for this toy dataset; neighborhood size is the main tuning parameter.
- ▶ `contamination='auto'`: expected outlier proportion and decision threshold.

```
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

X_lof = df_housing[['RM', 'MEDV']]
X_scaled = StandardScaler().fit_transform(X_lof)

lof = LocalOutlierFactor(n_neighbors=2)
yhat = lof.fit_predict(X_scaled)

mask = yhat != -1
print("Outliers:", sum(yhat == -1))
print(
    "Cleaned shape:",
    df_housing[mask].shape
)
```

Lab 2: Full Real-Estate Valuation Dataset

Load a full housing-market dataset with economically meaningful feature names.

```
column_names = [  
    "CRIM", # Per-capita crime rate  
    "ZN", # Residential land zoned over 25,000 sq.ft.  
    "INDUS", # Non-retail business acreage  
    "CHAS", # Charles River dummy variable  
    "NOX", # Nitric oxide concentration  
    "RM", # Average rooms per dwelling  
    "AGE", # Homes built before 1940 (%)  
    "DIS", # Distance to employment centers  
    "RAD", # Highway accessibility index  
    "TAX", # Property-tax rate  
    "PTRATIO", # Pupil-teacher ratio  
    "B", #  $1000(B_k - 0.63)^2$   
    "LSTAT", # Lower-status population (%)  
    "MEDV" # Median home value (USD thousands)  
]  
url_lab2 = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv"  
df_lab2 = pd.read_csv(url_lab2, header=None, names=column_names)  
df_lab2.head()
```

Part 3 – Handling Missing Data

PART 3 A Comprehensive Guide to Missing Values

The challenge

Most machine-learning algorithms cannot directly handle missing values. The treatment strategy can materially affect model performance.

Four strategies

- ▶ Removing data
- ▶ Statistical imputation
- ▶ KNN imputation
- ▶ Iterative imputation

Toy Dataset: Loan Applications

We use five applications with missing financial information.

```
csv_loans = '''application_id,income_k,debt_ratio,credit_score,loan_amount_k,default_flag
1001,60,0.30,720,100,0
1002,75,0.45,?,150,0
1003,80,?,680,120,1
1004,90,0.55,650,200,1
1005,70,0.35,700,?,0'''

df_loans = pd.read_csv(
    io.StringIO(csv_loans), na_values='?'
)
print(df_loans)
```

Business context

Missing credit scores, debt ratios, and requested loan amounts can affect approval and default-risk modeling.

Missing Data Removal: Concept and Hand Check

Concept

Complete-case analysis removes observations containing one or more missing values. Column removal discards an entire feature.

Pros and cons

Pro: quick and easy.

Cons: may discard substantial information and introduce bias.

Hand check on the toy data

Row	Missing count	Result
0	0	Retain
1	1	Remove
2	1	Remove
3	0	Retain
4	1	Remove

Complete rows = $2/5 = 40\%$; removing incomplete rows discards 60% of this sample.

Tool and Implementation: Missing-Data Removal

Pandas tools

- ▶ `isnull().sum()` audits missingness.
- ▶ `dropna(axis=0)` removes rows.
- ▶ `dropna(axis=1)` removes columns.

```
print("Missing values per column:")
print(df_loans.isnull().sum())

data_dropped = df_loans.dropna()
print("Data after dropping missing rows:")
print(data_dropped)
```

Statistical Imputation: Concept and Hand Calculation

Concept

Replace a missing value with a statistical summary such as the mean, median, or mode of its feature.

Trade-off

Simple and stable, but it reduces feature variance and ignores relationships with other features.

Mean imputation for credit_score

Observed values are

720, 680, 650, 700.

$$\bar{x} = \frac{720 + 680 + 650 + 700}{4} = 687.5$$

The missing credit score in row 1 becomes 687.5.

Tool and Implementation: SimpleImputer

Tool

`sklearn.impute.SimpleImputer`

Important parameters

- ▶ `strategy`: mean, median, most_frequent, or constant.
- ▶ `fill_value`: value used with the constant strategy.

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
loan_features = [
    'income_k', 'debt_ratio',
    'credit_score', 'loan_amount_k'
]
data_imputed_mean = imputer.fit_transform(
    df_loans[loan_features]
)

print(pd.DataFrame(
    data_imputed_mean,
    columns=loan_features
))
```

KNN Imputation: Concept and Hand Calculation

Concept

A multivariate method that finds the k most similar rows and imputes a missing value using the neighbors' average.

Practical requirement

Scale features first when their units differ substantially; otherwise large-scale variables dominate distance.

Toy calculation: impute credit score

The applicant's income is 75. The nearest complete-income records are

$$(70, 700), \quad (80, 680),$$

where each pair is (income, credit score). Both income distances equal 5. With $k = 2$ and uniform weights:

$$\widehat{score} = \frac{700 + 680}{2} = 690.$$

Tool and Implementation: KNNImputer

Tool

`sklearn.impute.KNNImputer`

Important parameters

- ▶ `n_neighbors=5`: number of neighbors.
- ▶ `weights='uniform'`: equal influence.
- ▶ `weights='distance'`: closer neighbors have greater influence.

```
from sklearn.impute import KNNImputer

knn_imputer = KNNImputer(
    n_neighbors=2,
    weights='uniform'
)
knn_cols = ['income_k', 'credit_score']
data_imputed_knn = knn_imputer.fit_transform(
    df_loans[knn_cols]
)

print(pd.DataFrame(
    data_imputed_knn,
    columns=knn_cols
))
```

Iterative Imputation: Concept and Hand Calculation

Concept

An advanced strategy that treats imputation as a machine-learning problem, modeling each incomplete feature as a function of the other features.

Iterative logic

Initialize missing entries, predict one incomplete feature from the others, update it, and repeat until estimates stabilize.

One simplified regression step

Suppose complete pairs (income, loan amount) are

(60, 100), (80, 140), (100, 180).

The fitted relationship is

$$\widehat{loan} = 2(income) - 20.$$

For income 75 and a missing loan amount:

$$\widehat{loan} = 2(75) - 20 = 130.$$

Tool and Implementation: IterativeImputer

Tool

`sklearn.impute.IterativeImputer`
Experimental API; enable it before importing.

Important parameters

- ▶ `estimator`: default `BayesianRidge`; other regressors can be used.
- ▶ `max_iter=10`: maximum imputation rounds.
- ▶ `random_state`: reproducibility.

```
from sklearn.experimental import
    enable_iterative_imputer
from sklearn.impute import IterativeImputer

iter_imputer = IterativeImputer(
    max_iter=10,
    random_state=0
)
data_imputed_iter = iter_imputer.fit_transform(
    df_loans[loan_features]
)

print(pd.DataFrame(
    data_imputed_iter,
    columns=loan_features
))
```

Lab 3: Loan-Default Portfolio with Missing Values

Generate 2,000 applications and inject 8% missingness into the predictors.

```
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=2000, n_features=8,
    n_informative=5, weights=[0.8, 0.2],
    random_state=42
)
feature_names = [
    'income_score', 'debt_score', 'liquidity_score',
    'payment_history', 'credit_utilization',
    'loan_size_score', 'business_age', 'collateral_score'
]
df_lab3 = pd.DataFrame(X, columns=feature_names)
rng = np.random.default_rng(42)
missing_mask = rng.random(df_lab3.shape) < 0.08
df_lab3 = df_lab3.mask(missing_mask)
df_lab3['default_flag'] = y
```

Lab goal

Compare row removal, SimpleImputer, KNNImputer, and IterativeImputer using retained rows and downstream AUC.

Part 4 – Automating with Pipelines

PART 4 Preventing Data Leakage

Data leakage

Information from outside the training data is used to create the model.

Example

Calculating the imputation mean on the entire dataset before train/test splitting.

Why it is dangerous

Leakage produces overly optimistic metrics and models that often fail in production.

Hand Calculation: How Leakage Changes Imputation

Incorrect: fit before splitting

Suppose training incomes are (20, 30, ?) and test incomes are (100, ?) thousand USD. Using all observed values gives

$$\bar{x}_{all} = \frac{20 + 30 + 100}{3} = 50.$$

The test value 100 influences preprocessing learned for training.

Correct: split, then fit on training

Learn the imputation value only from observed training values:

$$\bar{x}_{train} = \frac{20 + 30}{2} = 25.$$

Fill training and test missing values with 25, without using test information.

Why metrics change

The leaked value 50 makes the training transformation unrealistically aware of future data.

Tool: Scikit-learn Pipeline

What is a pipeline?

`sklearn.pipeline.Pipeline` chains transformers and a final estimator into one object. During `fit()`, transformations are learned only from training data.

```
from sklearn.pipeline import Pipeline

steps = [
    ('imputer', SimpleImputer()),
    ('model', RandomForestClassifier())
]
pipeline = Pipeline(steps)

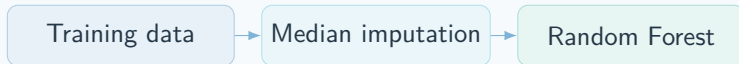
pipeline.fit(X_train, y_train)
```

Key benefit

The same fitted transformations are applied to test or production data without being re-fitted.

Components within the Pipeline

<code>train_test_split</code>	<code>SimpleImputer</code>	<code>RandomForestClassifier</code>
Splits data into training and testing subsets for honest evaluation on unseen observations.	Handles missing values first, ensuring complete inputs before modeling.	Builds multiple decision trees and aggregates their outputs for classification.



Implementation: Leakage-Safe Pipeline

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df_lab3_clean = df_lab3.dropna(
    subset=['default_flag']
)
X = df_lab3_clean.drop('default_flag', axis=1)
y = df_lab3_clean['default_flag']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('model', RandomForestClassifier(random_state=42))
])
```

```
# Training automatically performs:
# imputer.fit_transform(X_train)
# model.fit(X_train_transformed, y_train)
pipeline.fit(X_train, y_train)

# Evaluation automatically performs:
# imputer.transform(X_test), without refitting
# model.predict(X_test_transformed)
y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Pipeline has been trained.")
print(
    f"Accuracy on the test set: {accuracy:.4f}"
)
```

Correct behavior

Training learns imputation values; testing only applies them.

Summary and Key Takeaways

Start with the basics

Always check zero-variance features and duplicate observations first.

Choose the outlier method

Use IQR for robust univariate screening or LOF for multivariate local anomalies.

Select an imputation strategy

Progress from SimpleImputer to KNNImputer or IterativeImputer when justified.

Always use pipelines

Encapsulate preprocessing and modeling to prevent leakage and create production-ready code.

LECTURE 2

Q&A

Questions and discussion